

ARUSH JASUJA

New York, NY • (646) 208-7412 • aj4368@nyu.edu • [linkedin.com/in/arushjasuja](https://www.linkedin.com/in/arushjasuja) • github.com/arushjasuja

Systems engineer with 3 years making GPU-accelerated workloads fast in production: full-stack performance tracing, kernel-level optimization, compile-time code generation, and large-scale distributed training infrastructure. MS Computer Science, NYU. 5 peer-reviewed ML publications.

EDUCATION

New York University — MS, Computer Science | GPA: 3.83/4.0 | Sep 2024 – May 2026

Coursework: Graphics Processing Units (Prof. Mohamed Zahran), Operating Systems, Algorithms, Deep Learning (Prof. Yann LeCun)

Guru Gobind Singh Indraprastha University — B.Tech, Computer Science | CGPA: 8.0/10 | Sep 2016 – Sep 2020

EXPERIENCE

Lead Software Engineer | Ripik.AI — New Delhi, India | Aug 2022 – Jul 2024

- Traced deep learning workload bottlenecks across the full stack, from CUDA kernel execution to inter-node networking, using Nsight Systems and custom Python tooling that parsed Nsight Compute reports automatically. Findings decided which kernels and memory patterns to fix first and saved customers \$15M+ in deferred hardware spend.
- Cut a customer-facing inference pipeline from 14 hours to 2 hours across 8 manufacturing facilities: classified kernels as compute- vs memory-bound, fixed uncoalesced access (AoS to SoA layout), applied shared-memory tiling to the hot convolution kernels, and overlapped CPU preprocessing with GPU execution using CUDA streams and pinned memory. GPU throughput rose roughly 40% on unchanged hardware.
- Wrote a Python-to-CUDA-C++ code generator that produced 20+ kernel variants (tile sizes, dtypes, unroll factors) from one template with compile-time constants, shrinking 6,000 lines of copy-paste C++ to 600. A numerical regression it surfaced led to accumulation-aware test tolerances ($1e-5$ to $1e-7$) and a multi-step stability gate in staging.
- Built multi-node distributed training infrastructure in PyTorch: DDP with NCCL, then FSDP (ZeRO-3) with forward prefetch after tracing AllReduce to 58% of step time over inter-node links. Per-GPU model state dropped 96% (36.8 GB to 1.53 GB on 24 cloud GPUs); step time fell 27.7%.
- Mentored junior engineers on profiling methodology; one engineer's first solo optimization raised a kernel's memory throughput 3.2x against a CI-tracked performance baseline.

Software Engineer, AI/ML | Sequoia Insilico — New Delhi, India | May 2021 – Aug 2022

- Reduced CNN and LSTM inference latency 45% across 1M+ patient records using PyTorch profiling and INT8 post-training quantization, holding COVID-19 detection accuracy at the 90% clinical floor through class-stratified calibration.

PROJECTS

GPU-Optimized N-Body Simulation (NYU, Prof. Zahran's GPU course) | Dec 2025

- 6,095x speedup for a 100K-particle simulation on an RTX 4070, analysis-first: derived arithmetic intensity before writing CUDA (1.67 FLOP/byte vs 58 ridge, memory-bound by 35x), then verified each change against the model. AoS-to-SoA measured 4.26x against a ~3x sector-efficiency prediction; the gap traced to TLB fragmentation. Final kernels: 60% compute / 52% memory utilization.

Decoder-Only Transformer From Scratch — PyTorch and JAX (NYU Deep Learning) | 2025

- Implemented a GPT-style decoder twice, in PyTorch and JAX: multi-head attention with causal masking, RoPE, KV-cache inference, BPE tokenizer, and a data-parallel mixed-precision training loop. Cross-framework logit equivalence verified with NumPy oracles; unit tests cover attention masking and cache correctness.

RESEARCH PUBLICATIONS (5 total - <https://scholar.google.com/citations?user=Sc1Mkb8AAAAJ>)

"Connecting Dots with Deep Learning: Graph-Based Approach of Alzheimer's Conversion Prediction" | Alzheimer's & Dementia 2025 | Impact Factor: 11.1

"Layer-wise Adaptive Sine Activation Based Recurrent Network for MCI Conversion" | Alzheimer's & Dementia 2025 | Impact Factor: 11.1

TECHNICAL SKILLS

Languages: C++ (CUDA C/C++), C, Python, SQL, Bash

Systems: GPU architecture, memory hierarchy, warp scheduling, occupancy tuning, roofline and Speed-of-Light analysis, profiling (Nsight Compute/Systems), compile-time code generation and kernel specialization, large-scale distributed systems, Linux

Parallel & distributed: CUDA kernel development, parallel algorithm design, NCCL, distributed training (DDP, FSDP/ZeRO), collective-communication profiling, memory bandwidth optimization (coalescing, shared-memory tiling)

ML stack: PyTorch, JAX, TensorFlow, TensorRT, mixed precision (FP16/BF16), quantization (INT8), data structures and algorithms

Leadership: Mentored 15+ engineers; speaker at 3 conferences; NYU course section leader (Architecture, Algorithms)